ChatGPT

Unified Textile Weaving Software Architecture

Designing a scalable textile weaving system requires a shared data model and a modular architecture. We combine three subsystems (design generator, feeder mapper/visualization, and user design editor) by defining common data structures and clear module boundaries. Below is a detailed breakdown of each research objective.

1. Unified Data Model

The heart of the system is a **central weaving model**. We represent each cloth design with structured data for:

- **Draft (Threading) plan:** mapping each warp *end* to a *shaft* (an array `warpToShaft[end] = shaftID`).
- **Lift/Treadle (Peg) plan:** for each pick (row), which shafts lift (a 2D binary or bitset `shaftLift[pick][shaft]`).
- **Design matrix:** a binary grid (`designMatrix[pick][end]`) marking whether warp is above weft. This is the "pattern" [1].
- **Feeder sequence:** an array `feederSeq[pick]` assigning a yarn feeder (color) to each pick.
- **Color palettes:** e.g. arrays mapping warp ends or feeders to color indices.

This model handles raw (binary) and colorized views. For instance, a **color matrix** is derived by merging `designMatrix` with `feederSeq` and palette data (warp vs weft color per cell). To support repeats/motifs, we also include repeat descriptors (e.g. `repeat = { width, height }`).

In practice we use *multiple domain models* for efficiency: a high-level "design model" (nested patterns, groups, repeats) and a flattened "draft model" (linear arrays) [2]. For example, user edits or generation might operate on a symbolic pattern (with repeats), then we build a raw grid for rendering. The F# domain-driven design example illustrates this: the **design model** keeps constructs like groups/repeats, while a separate **draft model** unrolls them into flat arrays for fast drawing [2]. Having two models means each can be optimized: the domain model uses domain vocabulary and the draft model uses primitive arrays for speed [2]. Conversion functions (`buildDraftModel(design)`) sync them as needed.

Example data schema (JSON) might look like:

```
{
  "shafts": 4, "ends": 12, "picks": 30,
  "warpToShaft": [1,2,3,4,1,2,3,4,1,2,3,4],
  "liftPlan": [ [1,0,0,1], [0,1,1,0], … ],        // each row = shafts up/down
  "designMatrix": [ [0,1,0,1,…], [1,0,1,0,…], … ], // 2D binary grid
  "feederSeq": [0,1,0,1, …],                       // feeder ID per pick
  "warpColors": [0,0,1,1,…],                       // index into color palette
```

```
  "feederColors": [1,0,2,0,...],                    // index into palette per
feeder
  "repeat": { "width": 12, "height": 30 }
}
```

This single JSON (or class) carries all info; modules read/write fields consistently. Data types like bitsets or sparse arrays may be used internally for efficiency. The key is **one shared model** so every subsystem (generator, editor, mapper) refers to the same fields.

Importantly, design vs. draft models **coexist and map between each other**. The F# blog notes "don't try to make one domain model work for every purpose… create more than one domain model… then map or convert between them" [2]. We adopt that: one model for editing/generation, another for visualization/drawing, kept in sync via conversion routines. This preserves domain vocabulary (e.g. repeats, named groups) while giving fast access for UI and output.

2. System Architecture

A modular design is key. We recommend a **layered, container-based architecture**: a rich frontend client and one or more backend services (see figure below).

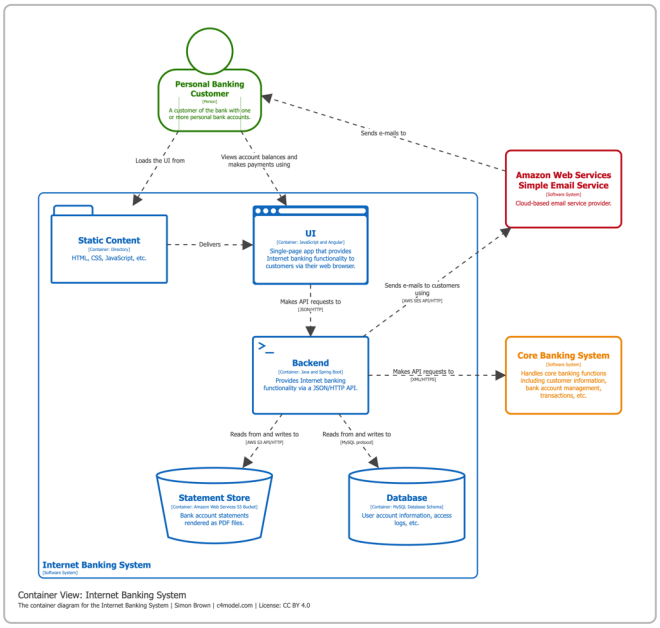

Figure: Example container-level architecture diagram. A client UI (green) communicates via a REST/GraphQL API (teal) to backend services (blue) like Design Engine and Feeder Engine, with a shared database (cylinder) and optional external services (gray).

- **Frontend (UI):** A single-page or desktop app providing the design tools and visualization. It includes the custom design editor (pixel tools, shape tools), repeat controls, and the color-coded grid view. It maintains local state (current design matrix, draft plan, etc.) and reacts to user edits or results from

backend computations. State management (e.g. Redux, MobX, or observer patterns) ensures all views (matrix, draft, color) stay in sync.
- **Backend/Logic engine:** One service or a set of services implementing the heavy logic:
- *Design Generation Engine:* Given inputs (shafts, ends, constraints), enumerates or computes weave patterns. This can be CPU-intensive (combinatorics), so it may run on a server (possibly containerized/microservice) or as a WebAssembly/worker in the client.
- *Feeder Mapping Engine:* Assigns feeder IDs to picks and applies colors; computes color matrices and machine instructions. This is less heavy but benefits from consistency checks.
- *Optional modules:* e.g. simulation, optimization, AI suggestion engines (discussed later).
- **Data/API layer:** A well-defined API (REST or GraphQL) connects UI and backend. For example: `POST /api/generate` returns a design matrix; `GET /api/design/{id}` fetches a saved design; `POST /api/feeder-map` applies a feeder pattern. Real-time needs could use WebSockets or server-sent events for updates (e.g. progress of generation).
- **Database/Storage:** Stores design objects, user presets, machine configurations. Could be an SQL/NoSQL DB with tables/collections for designs, palettes, etc.
- **Visualization/Simulation Layer:** A module (either frontend or separate service) for rendering machine-specific previews (e.g. 3D weave simulation or loom peg charts).

This **containerized approach** balances modularity and performance. Each component can be scaled or updated independently. For a smaller scope, a modular monolith (one codebase with clear package boundaries) is possible too. The key is **loose coupling and clear interfaces**. For example, like in traditional EDA CAD (P-CAD), separate modules communicate via well-defined files: P-CAD's schematic and PCB tools exchange a netlist text file [3]. Similarly, our subsystems can interchange data (e.g. JSON payloads, message queues) without tight integration.

3. Pipeline Design

The user/workflow pipeline is a sequence of steps:

1. **Input:** User selects mode – either run the *Design Generation Engine* (with parameters like shafts, ends, constraints) or use the *Custom Design Editor* (drawing tools).
2. **Design Creation:**
3. *Auto-Generated:* The generator computes possible weave matrices (potentially asynchronously).
4. *User-Drawn:* The editor updates the design matrix in real time as the user draws or pastes shapes. It also applies the defined repeat pattern to tile the motif.
5. **Draft Plan Integration:** If not already included, the system generates or updates the draft/tie-up plan (threading + lift plan). Often user or presets define the threading, and the lift plan (peg sequence) can be derived from the generated design or vice versa.
6. **Feeder Mapping:** The feeder engine takes the binary design and assigns feeder IDs to each pick row (`feederSeq`). It then produces a color-coded version of the matrix (using warp colors for "1" cells and weft colors for "0" cells according to the feeder's yarn color).
7. **Visualization/Output:** The final matrices and plans feed into visualization modules:
8. A **color grid** display in the UI,
9. A **machine preview** (e.g. color weave draft, peg chart),
10. Export to **machine code** (e.g. CSV or proprietary format for the loom).

Throughout, the UI shows progress or updates (especially during heavy generation). For real-time feedback, we design the flow to be **event-driven**: user edits fire events updating the shared state; async engine results publish events back to the frontend.

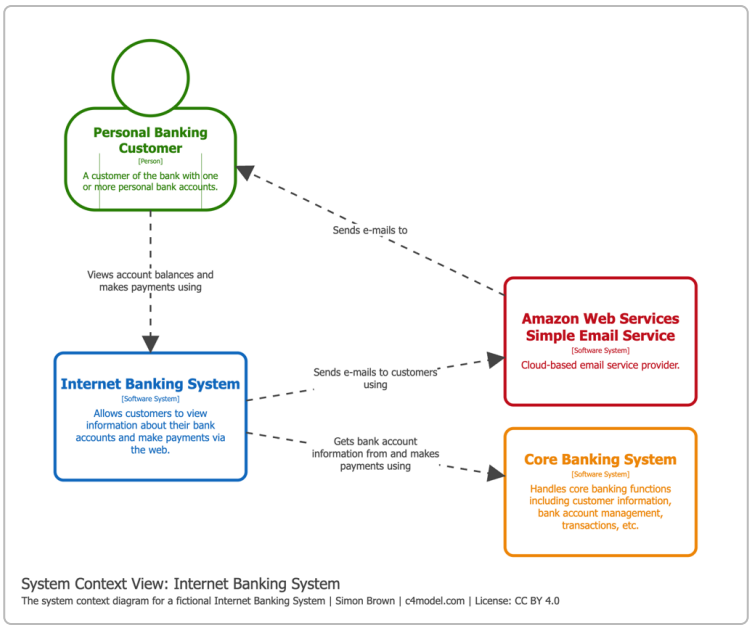

Figure: Example context/data-flow diagram. User and external systems (gray) interface with the WeavingApp (green). Data flows: User actions → frontend UI → API → engines (design, feeder) → updated state/visuals.

A data-flow diagram for the pipeline helps visualize this: the UI calls `/generate` or `/design` APIs, receives a new matrix, then calls `/mapFeeder`. Alternatively, the UI may run simple logic itself (e.g. for repeating motifs) without server round-trips for snappiness. Long-running tasks (exhaustive design generation) run on the backend with async callbacks.

Step-by-step integration logic:
- UI dispatches a **"GenerateDesign"** command to the backend (or spawns a worker).
- Backend returns a design matrix (or multiple candidates).
- UI commits the matrix to state, triggers rerender of draft and color views.
- User or system defines `feederSeq`; UI or backend maps it to colors.
- The full design object is updated and, if saved, persists to DB.

This pipeline ensures that all three subsystems can operate on the shared data. For example, when the generator finishes, it produces exactly the same design-matrix format that the editor and feeder modules consume, so everything "just works" without adhoc translations.

4. State Management

Managing the weaving state is crucial. We maintain a **single source of truth** for the design state (matrices, plans, feeders, repeats). In a web UI, a state container (Redux/Vuex/MobX) or React Context can hold this. Key state elements include: the *design matrix*, *threading plan*, *lift plan*, and *feeder sequence*.

- **Immutable updates:** To support undo/redo and predictable rendering, use immutable data structures or copy-on-write updates.
- **Reactivity:** Whenever the design matrix changes (user draw, generation result, repeat size change), derived elements (draft grid, color grid) automatically recompute. This is the "draft model" of [49], built from the domain model.
- **Synchronization:** If the user switches between editor and generator, the state syncs both ways. For instance, editing threading will recompute the draft view immediately.
- **Concurrency:** If backend tasks run asynchronously, responses update the state via dispatched actions/events. We must manage in-flight state (e.g. disable some UI until generation completes).

Importantly, we treat each subcomponent of the state as linked. For example, changing the threading affects which shafts rise for each row, so we must recalc the design matrix or at least the visualization. Techniques like the Observer pattern or a reactive stream (RxJS) can auto-propagate such changes. This mirrors how the F# example separates the **design domain model** from the **draft model**: the draft is rebuilt whenever the design changes [2].

5. UI/UX Integration

The user experience ties together design creation, exploration, and machine preview smoothly. Key considerations:

- **Single-Page Flow:** The interface can be organized into panels or tabs (e.g. *Design Editor*, *Pattern Explorer*, *Machine Preview*), but they share state. Transitions (e.g. generating a design then immediately previewing it) should feel seamless.
- **Real-Time Feedback:** As the user draws or edits the design grid, they should see the draft plan and color mapping update live. Tools like Fabric.js or HTML5 Canvas with WebGL can render large grids efficiently.
- **Visual Aids:** Provide grid zooming, panning, and pattern repeat overlays. For example, showing outlines of the repeat block or color legends for feeders.
- **Interactivity:** Clicking on a grid cell toggles warp/weft; dragging adds shapes. Assigning feeders could be a drag-and-drop of color swatches onto rows.
- **Validation/Warnings:** If the design violates loom constraints (e.g. too many shafts raised at once), highlight warnings.
- **Contextual Help:** The UI can display the draft (threading and lift plan) and peg chart side-by-side with the drawn pattern, helping users understand the structure.

From a code standpoint, the UI invokes the backend (or local algorithm) for generation and feeder mapping. It then consumes the resulting matrices to update visual components. State flows down as props or context. Libraries like React or Vue facilitate this unidirectional data flow.

6. Performance Optimization

Handling large patterns (e.g. 1000×1000) demands care:

- **Virtualized Rendering:** Do not render all 1,000,000 cells at once in the DOM. Use canvas or GPU-based rendering. If using HTML table/divs, only render the visible viewport (windowing).
- **Efficient Data Structures:** Store the matrix in a compact form (bit arrays, typed arrays). For binary data, a `Uint8Array` or bitset is much smaller than a nested Array of booleans.
- **Incremental Updates:** When a user edits a small part of the design, update only affected pixels in the canvas. Avoid re-drawing the entire grid on every change.
- **Offload Heavy Computation:** The combinatorial generation engine may run on a backend server or a Web Worker. This keeps the UI responsive. Backends can use multi-threading or optimized libraries for graph algorithms.
- **Caching/Throttling:** If the user changes parameters rapidly (e.g. scrolls the repeat size slider), debounce or cache results.
- **Parallel Feeder Mapping:** Mapping a feeder sequence to color involves scanning the matrix – this is O(n) and can be parallelized on CPU or GPU if needed, but is usually fast enough in native code.
- **Memory Management:** Clear unused data when designs are closed. For example, free the matrix array when switching projects.

In short, treat the design matrix like image data: use graphics techniques. Many data-grid libraries (e.g. **Webix DataTable** or **React Virtualized**) offer high-performance handling of large datasets and should be considered. Even so, custom canvas drawing will likely be necessary for ultimate speed and flexibility.

7. Extensibility

To future-proof the system, design with extensibility in mind:

- **Plugin-friendly architecture:** Define clear interfaces for modules so new features can "plug in". For example, a simulation engine could subscribe to an event like `onDesignUpdate`. Techniques like dependency injection or a service locator allow adding new components without modifying core code.
- **Versioned Data Model:** As features evolve (AI suggestions, multi-layer weaves, new loom types), the data model can change. Include version tags in saved designs so code can migrate old data formats.
- **API Layer:** Expose internal functionality through APIs. For example, a REST endpoint `/api/suggest` could be implemented later to provide AI-driven design suggestions without altering the frontend.
- **Separation of Concerns:** Keep UI decoupled from business logic. Future features like *fabric simulation* or *optimization* can be new services that read the same design state.
- **Open Data Formats:** Use standard or documented formats (JSON, XML) for import/export. This makes it easy to interoperate with other CAD tools.
- **Configurable Workflows:** Allow enabling/disabling modules (e.g. turn off real-time mapping for extremely large designs).

By following patterns like **Command Query Responsibility Segregation (CQRS)** or **Event Sourcing**, the system can record user actions and potentially "replay" them in new ways (useful for AI/analytics). In

essence, think of the system as a *platform* that can incorporate weaving-specific AI, optimization, or simulation tools as separate microservices or libraries.

8. Real-World CAD/Software Patterns

Existing CAD systems offer useful lessons. For instance, many CAD tools separate modeling and rendering. As noted, P-CAD uses distinct schematic and PCB modules that exchange a netlist file [3]. Similarly, a modern weaving CAD might export an intermediate file (or JSON) as the interface between design and loom modules.

Popular weaving CAD (e.g. ArahWeave, NedGraphics, Pointcarre) often use plugin/plugin frameworks and tiered architectures (UI → core engine → output). While proprietary, they emphasize: - **Layered MVC-like design:** separate controllers for data vs. views. - **Template/customizable modules:** e.g. allowing user-defined scripts or macros. - **Vector/bitmap hybrid graphics:** for drawing patterns and previews.

Best practices from software architecture also apply (C4 model, microservices patterns, etc.). For example, our system's container diagram (above) and data-flow approach mirror the C4 methodology of clear system context, containers, and components. In practice, many CAD products adopt a **modular monolith** (single application with clear subsystems) or a thin client + rich server model.

In summary, the system should look like this: the UI is a thin **view/controller** layer; the core weaving logic (the "model") lives in backend modules. Data flows in one direction (user → design data → visual output), and new features can hook into this flow without breaking existing code. By using a unified data schema and well-defined APIs, all parts—generation, editing, mapping, and visualization—"speak the same language." This ensures a cohesive, production-ready system that can grow with new weaving technologies [2] [3].

---

[1] Plain Weave Drafting and Design Guide | PDF | Weaving | Loom
https://www.scribd.com/document/548968463/Module-2-Ref-1-Fundamentals-Design-Draft-and-Peg-Plan

[2] Designing weaving software | F# for fun and profit
https://fsharpforfunandprofit.com/posts/designing-weaving-software/

[3] iarjset.com
https://iarjset.com/wp-content/uploads/2018/06/IARJSET-1.pdf